GAME.md — симулятор админа ЦОД (рабочее название: «Uptime Empire»)

Главный документ проекта. ИИ-агент читает его ПЕРВЫМ в каждой сессии, затем DESIGN.md.

1. Концепция

Экономический симулятор (тайкун) для **ПК (Windows и Linux)**. Веб-версия пока не планируется. Игрок начинает с **домашнего сервера** в комнате, покупает комплектующие, собирает серверы, продаёт услуги клиентам (сайты, чат-боты, игровые серверы, VPS и т.д.) и вырастает до **огромного ЦОД** с многомиллионной прибылью.

- Один игрок, без мультиплеера, без сюжета. Цель задают вехи роста (DESIGN.md, разд. 10).
- **Графика рисуется кодом**: тёмный «терминальный/неоновый» интерфейс, стойки и серверы SVG-фигурами, графики. Никаких внешних картинок, карт и моделей.
- Игровая логика реалистична по смыслу (ядра, память, диски, трафик, питание, охлаждение, аптайм), но упрощена. Все бренды вымышленные.
- Длительность прохождения: ориентир 15–25 часов.

2. Технологии

- **Платформа: только ПК.** Оболочка **Electron** (процессы `main` и `preload`, интерфейс в окне) + **TypeScript (strict) + Vite + React + Zustand**. Тесты: Vitest. Упаковка: `electron-builder` (Windows: установщик и portable, Linux: AppImage).
- Системные вещи (файлы сохранений, окно, полный экран, логи) только через слой `src/platform/` и безопасный IPC. Интерфейс не имеет прямого доступа к Node.js (`contextIsolation: true`, `nodeIntegration: false`).
- Графики: собственные SVG-компоненты или `uPlot` (по согласованию). Другие библиотеки только после согласования с человеком.
- Сохранения и настройки: файлы JSON в папке `userData` ОС, с версией и миграциями.
- Все тексты через `src/i18n/` (ru, en) с самого начала. Шрифты только с открытой лицензией (OFL).
- Звук (позже): генерируется через WebAudio, без внешних файлов.
- Позже по желанию: Steam (достижения, облачные сохранения) через слой `platform/`.

3. Правила для агента (обязательно)

0. Перед работой прочитай **GAME.md и DESIGN.md** целиком.
1. Работай **по одному этапу** из раздела 7. Следующий без команды не начинай.
2. Сначала кратко изложи план этапа (5–10 строк), затем пиши код.
3. После этапа: `npm run build` и `npm test` проходят.
4. Делай git-коммит после каждого завершённого шага: `stage N: описание`, затем `git push` в `origin` (если remote настроен). Никогда не выводи, не читай и не коммить токены, ключи и `.env`.
5. Обнови чек-лист в разделе 7 и `CHANGELOG.md`.
6. В конце сообщи: что сделано, как запустить, **что человеку проверить руками** (3–7 пунктов).
7. Не ломай то, что работает: перед правкой старого кода прогоняй тесты.
8. Логика (`src/sim/`) не импортирует React и DOM. Нарушение это ошибка.
9. **Все числа баланса** живут в `src/data/*.json`, не в коде. Строки интерфейса только через i18n.
10. Если требование неясно, задай один конкретный вопрос. Мелкие решения записывай в `DECISIONS.md`.
11. Не добавляй фичи вне текущего этапа, идеи записывай в `IDEAS.md`.
12. Начиная с этапа 6 (баланс-симулятор), после каждого этапа, влияющего на экономику, **запускай** `npm run balance-sim` и сверяй результат с таблицей темпа (DESIGN.md, разд. 10). Расхождение больше 30% чини балансом, а не кодом.
13. Следи за бюджетом производительности (DESIGN.md, разд. 13).
14. **Безопасность Electron:** никакого `remoteModule`, никакой загрузки внешних сайтов в окне игры, все обращения к файлам только через `preload` и ограниченный набор IPC-каналов (`save:*`, `settings:*`, `log:*`).
15. Перед каждым этапом пробегись по разделам 15–21 DESIGN.md и реализуй нужные мелочи, относящиеся к этапу, а не откладывай их.

4. Архитектура

```
electron/     # main-процесс (окно, меню, IPC), preload
src/
  sim/        # чистая логика, БЕЗ React и Electron: время, экономика, железо,
клиенты, события
  ui/         # React-компоненты и экраны
  state/      # Zustand-хранилище, связка sim и ui
  platform/   # доступ к системе (сохранения, настройки, окно) через IPC
  data/       # components.json, services.json, events.json, research.json,
locations.json, staff.json, balance.json
  save/       # сериализация, миграции версий
  i18n/       # ru.json, en.json
```

```
  tools/      # balance-sim, генерация отчётов
tests/        # Vitest для sim/
```

Принципы:

- Симуляция **дискретная**: один тик = один игровой час. Сущности хранятся агрегированно (группы однотипных серверов, пулы клиентов), чтобы поздняя игра с тысячами серверов не тормозила.
- Детерминизм: вся случайность через seeded RNG (`core/rng`), `Math.random` запрещён.
- События через шину (`serverFailed`, `customerJoined`, `monthClosed`), системы не вызывают друг друга напрямую.
- Состояние игры полностью сериализуется в JSON.

5. Ядро игры (кратко, детали в DESIGN.md)

Цикл: **купить железо → собрать сервер → разместить → подключить сервисы → клиенты платят → расширяться**.

Эпохи игры:

1. **Комната:** старый ПК, домашний интернет, шум и перегрев.
2. **Гараж:** первая стойка, ИБП, кондиционер, выделенный канал.
3. **Арендованный зал (colo):** несколько стоек, персонал, SLA-контракты.
4. **Свой ЦОД:** здание, электрощит, генераторы, чиллеры, уровни надёжности.
5. **Корпорация:** несколько ЦОД, облако, GPU-кластеры, CDN.

6. Вне объёма (пока)

Веб-версия и мобильное управление, мультиплеер, настоящий 3D, сюжет, реальные бренды, незаконные «услуги», внешние картинки и звуки, процедурные карты.

7. Дорожная карта (чек-лист)

Каждый этап играбелен и закрывается коммитом.

- [ ] **0. Каркас.** Electron + Vite + TS + React + Zustand, `electron/` и `src/platform/`, i18n, Vitest, игровой цикл и время (скорости, пауза), отладочный оверлей F3, `CHANGELOG.md`. *Готово, когда:* `npm run dev` открывает окно приложения с часами, `npm test` зелёный.
- [ ] **1. Меню, настройки и сохранения.** Главное меню, новая игра (сложность), слоты сохранений и автосохранение, пауза, настройки (видео, звук, управление, игра, доступность), горячие клавиши.
- [ ] **2. Экономика и время.** Деньги, ежемесячные счета, электричество и интернет, дашборд, стартовое состояние.

- [ ] **3. Железо.** Каталог комплектующих, сборка, совместимость, магазин новых и б/у, доставка, склад запчастей, износ.
- [ ] **4. Дом и размещение.** Домашний сервер: питание, интернет, нагрев, шум, соседи.
- [ ] **5. Клиенты и сервисы.** Каталог услуг, тарифы игрока, спрос, заселение клиентов, оверселлинг, доход, отток, счета и просрочки.
- [ ] **6. Баланс-симулятор.** `npm run balance-sim`, бот-стратегия, кривая роста, настройка темпа по таблице.
- [ ] **7. Репутация и аптайм.** Расчёт аптайма, SLA, тикеты поддержки, репутация и отзывы.
- [ ] **8. События и аварии.** Отказы, перегрев, отключение света, DDoS, абьюз клиентов, резервные копии и RAID, окна обслуживания.
- [ ] **9. Гараж.** Инфраструктура 2 уровня: стойка, ИБП, кондиционер, выделенный канал, переезд.
- [ ] **10. Персонал и автоматизация.**
- [ ] **11. Исследования.**
- [ ] **12. Colo и масштабирование.** Арендованный зал, много стоек, PDU, горячие точки, PUE, резервирование питания.
- [ ] **13. Рынок и конкуренты.** Конкуренты, ценовые войны, маркетинг, контракты и крупные клиенты.
- [ ] **14. Поздняя игра.** Собственный ЦОД, регионы, уровни надёжности Tier, GPU и облако, CDN, корпорация.
- [ ] **15. Интерфейс, подсказки, звук, обучение.** Полировка экранов, контекстные подсказки, обучающий сценарий первых 20 минут, звук (WebAudio).
- [ ] **16. Оптимизация и оффлайн-прогресс.** Профилирование, бюджеты, догоняющий расчёт, достижения и статистика.
- [ ] **17. Упаковка для ПК.** Установщик Windows и portable, AppImage для Linux, иконка, проверка запуска на чистой системе, страница на itch.io.

8. Роль человека

1. **Окружение:** Node.js (LTS), Git, редактор кода, OpenCode с моделью, GitHub-репозиторий.
2. **Тестирование:** после каждого этапа играть ~5–10 минут и писать замечания: понятно ли, интересно ли, не слишком ли быстро или медленно растут деньги.
3. **Баланс и решения:** название, насколько жёстко наказывать за ошибки, какие услуги оставить.
4. **Проверка лицензий:** шрифты и всё, что подключаешь извне.
5. **Проверка на своей системе:** запускать установленную сборку (не только `npm run dev`), проверять сохранения, полный экран, масштаб интерфейса, нагрузку на процессор.
6. **Публикация позже:** аккаунт itch.io, страница игры, видео о разработке. Неподписанные установщики Windows могут вызывать предупреждения SmartScreen и антивируса, это

нормально, а для снижения жалоб позже понадобится подпись кода.

9. Проверки качества

- Тесты `sim/`: экономика, совместимость комплектующих, отказы и аптайм, спрос и отток, сериализация.
- Баланс-симулятор после каждого этапа с экономикой (разд. 3, правило 12).
- Производительность: плавный интерфейс при 5000 серверов и 50 000 клиентов в агрегированной симуляции.
- Дымовой тест собранного приложения: запускается, создаёт и загружает сохранение, закрывается без ошибок в логах.